

CZ4052/SC4052 Cloud Computing

Semester 2 Examination 2024–2025 — Model Answers

Question 1 — TCP AIMD Throughput Analysis (25 marks)

Background

The sawtooth AIMD algorithm increases the congestion window W by 1 MSS per RTT (additive increase) until a packet loss occurs, then halves W (multiplicative decrease). Let W be the window size **at the moment of packet loss**.

(a) Express STT in terms of RTT and W (5 marks)

After each loss event, the window is halved:

$$W_{\min} = W/2$$

The window then grows linearly by 1 MSS per RTT from $W/2$ back up to W . The number of RTTs required is:

$$\text{Number of RTTs} = W - W/2 = W/2$$

Therefore the **Sawtooth Time (STT)** — the time between consecutive packet losses — is:

$$STT = \frac{W}{2} \times RTT$$

(b) Express packet loss rate p in terms of W (10 marks)

In one sawtooth period, the window ranges from $W/2$ to W . The average window size over this period is:

$$\bar{W} = \frac{W/2 + W}{2} = \frac{3W}{4} \text{ MSS}$$

The number of RTTs per STT is $W/2$ (from part a). So the total number of packets sent per STT is:

$$\text{Packets per STT} = \bar{W} \times \frac{STT}{RTT} = \frac{3W}{4} \times \frac{W}{2} = \frac{3W^2}{8}$$

Since exactly **one packet loss** occurs at the end of each STT:

$$p = \frac{\text{losses per STT}}{\text{packets per STT}} = \frac{1}{3W^2/8}$$

$$p = \frac{8}{3W^2}$$

Equivalently: $W = \sqrt{\frac{8}{3p}}$

(c) Average TCP Throughput as a function of RTT, MTU, and p (10 marks)

Average throughput = (average bytes sent per RTT) / RTT

$$\text{Average bytes per RTT} = \bar{W} \times MTU = \frac{3W}{4} \times MTU$$

Substituting $W = \sqrt{8/(3p)}$:

$$\text{Throughput} = \frac{3W}{4} \times \frac{MTU}{RTT} = \frac{3MTU}{4 \cdot RTT} \times \sqrt{\frac{8}{3p}}$$

Simplify:

$$= \frac{3MTU}{4 \cdot RTT} \times \frac{2\sqrt{2}}{\sqrt{3p}} = \frac{3 \times 2\sqrt{2} \cdot MTU}{4\sqrt{3} \cdot RTT \cdot \sqrt{p}} = \frac{6\sqrt{2} \cdot MTU}{4\sqrt{3} \cdot RTT \cdot \sqrt{p}}$$

$$= \frac{3\sqrt{2}}{2\sqrt{3}} \times \frac{MTU}{RTT\sqrt{p}} = \sqrt{\frac{9 \times 2}{4 \times 3}} \times \frac{MTU}{RTT\sqrt{p}}$$

$$\text{Throughput} = \sqrt{\frac{3}{2}} \times \frac{MTU}{RTT\sqrt{p}} \approx \frac{1.22 \cdot MTU}{RTT \cdot \sqrt{p}}$$

Interpretation: TCP throughput degrades proportionally to $1/\sqrt{p}$. Doubling the packet loss rate reduces throughput by $\approx 30\%$. Halving the RTT doubles throughput.

Question 2 — RAID5 and EDF Scheduling (25 marks)

(a) RAID5 Minimum Redundancy + Comparison (10 marks)

Minimum Redundancy for Single-Failure Recovery An (n, k) erasure code encodes k data blocks into n total blocks, where $r = n - k$ are redundancy blocks. To recover from at most 1 failure:

- Any k surviving blocks must allow reconstruction of the n -th missing block.
- One parity block (XOR of all k data blocks) is **sufficient** to recover any single failure.

$$r_{\min} = 1$$

This gives $n = k + 1 = 5$ blocks total.

Probability of Data Loss with k=4, p=0.002 With $n = 5$, $r = 1$: data is lost if **2 or more** blocks fail simultaneously.

$$P(\text{loss}) = \sum_{i=2}^5 \binom{5}{i} p^i (1-p)^{5-i}$$

For small p , the dominant term is $i = 2$:

$$P(\text{loss}) \approx \binom{5}{2} p^2 = 10 \times (0.002)^2 = 10 \times 4 \times 10^{-6} = 4 \times 10^{-5}$$

Comparison with 2-Replication In 2-replication, every data block is stored twice ($n = 2k = 8$ blocks, $r = k = 4$). Data is lost if **both** copies of any single block fail.

$$P(\text{loss per block}) = p^2$$

$$P(\text{system loss}) \approx 1 - (1 - p^2)^4 \approx 4p^2 = 4 \times (0.002)^2 = \mathbf{1.6 \times 10^{-5}}$$

Scheme	n blocks	Unique data	Storage overhead	$P(\text{loss})$
(5, 4) erasure code	5	$4S$	25%	$\approx 4 \times 10^{-5}$
2-replication	8	$4S$	100%	$\approx 1.6 \times 10^{-5}$

Conclusion: The (5, 4) erasure code achieves comparable failure resilience to 2-replication while using only 25% storage overhead vs 100%. For very low p , both are adequate; the erasure code is more storage-efficient.

(b) EDF Scheduler — Schedulability and Deadline Analysis (15 marks)

Tasks: (4, 10, 0), (3, 8, 0), (2, 6, 0), (5, 15, 0), (6, 20, 0)

Step 1: Processor Utilization Check

Task	x (CPU)	y (period)	$u_i = x/y$
T1	4	10	0.400
T2	3	8	0.375
T3	2	6	0.333
T4	5	15	0.333
T5	6	20	0.300
Total			$U = 1.741$

Since $U = 1.741 > 1$, the task set is **not schedulable** under EDF (or any preemptive algorithm).

Step 2: Earliest Deadline Miss Trace EDF execution from $t = 0$ (deadlines: T1→10, T2→8, T3→6, T4→15, T5→20):

t	Runnable tasks (deadline, remaining)	EDF choice
0	T3(6,2), T2(8,3), T1(10,4), T4(15,5), T5(20,6)	T3
1	T3(6,1), T2(8,3), T1(10,4), T4(15,5), T5(20,6)	T3 done
2	T2(8,3), T1(10,4), T4(15,5), T5(20,6)	T2
3	T2(8,2), T1(10,4), T4(15,5), T5(20,6)	T2
4	T2(8,1), T1(10,4), T4(15,5), T5(20,6)	T2 done
5	T1(10,4), T4(15,5), T5(20,6)	T1
6	T3(12,2) [†] , T1(10,3), T4(15,5), T5(20,6)	T1

t	Runnable tasks (deadline, remaining)	EDF choice
7	T3(12,2), T1(10,2), T4(15,5), T5(20,6)	T1
8	T3(12,2), T1(10,1), T2(16,3)†, T4(15,5), T5(20,6)	T1 done
9	T3(12,2), T2(16,3), T4(15,5), T5(20,6)	T3
10	T3(12,1), T1(20,4)†, T2(16,3), T4(15,5), T5(20,6)	T3 done
11	T1(20,4), T2(16,3), T4(15,5), T5(20,6)	T4
12	T1(20,4), T2(16,3), T3(18,2)†, T4(15,4), T5(20,6)	T4
13	T1(20,4), T2(16,3), T3(18,2), T4(15,3), T5(20,6)	T4
14	T1(20,4), T2(16,3), T3(18,2), T4(15,2), T5(20,6)	T4
15	T1(20,4), T2(16,3), T3(18,2), T4(15,1), T5(20,6)	T4 (runs slot [15,16), completes at $t = 16$)

† = new period begins

T4's deadline is $d = 15$ but its 5th and final execution unit runs during [15, 16), completing at $t = 16$.

First deadline miss: Task T4 at $t = 15$ (completes at $t = 16 > d = 15$)

Step 3: Suggested Modification Reduce CPU demands to bring $U \leq 1$ without removing any task. One feasible modification:

Task	Original (x, y, z)	Modified (x, y, z)	u_i original	u_i modified
T1	(4, 10, 0)	(2, 10, 0)	0.400	0.200
T2	(3, 8, 0)	(1, 8, 0)	0.375	0.125
T3	(2, 6, 0)	(2, 6, 0)	0.333	0.333
T4	(5, 15, 0)	(5, 15, 0)	0.333	0.333
T5	(6, 20, 0)	(1, 20, 0)	0.300	0.050
Total			1.741	1.041

Hmm, still slightly over. Reduce T3 further:

Change T3 to (1, 6, 0): $u_3 = 0.167$. New total = $0.200 + 0.125 + 0.167 + 0.333 + 0.050 = 0.875 \leq 1$

Suggested feasible task set:

Task	Modified (x, y, z)	u_i
T1	(2, 10, 0)	0.200
T2	(1, 8, 0)	0.125
T3	(1, 6, 0)	0.167
T4	(5, 15, 0)	0.333
T5	(1, 20, 0)	0.050

Task	Modified (x, y, z)	u_i
Total		$U = 0.875 \leq 1$

With $U = 0.875 \leq 1$, EDF can schedule all tasks and meet every deadline. Alternatively, one could increase task **periods** (relax timing requirements) rather than reducing CPU time — for example, changing T2 from $(3, 8, 0)$ to $(3, 24, 0)$ reduces u_2 to 0.125, freeing substantial CPU budget.

Question 3 — RSA Digital Signature (25 marks)

(a) RSA Key Setup: $p=3, q=11, e=7$ (5 marks)

Modulus:

$$n = p \times q = 3 \times 11 = 33$$

Euler's Totient:

$$\phi(n) = (p-1)(q-1) = 2 \times 10 = 20$$

Private key exponent d : Find d such that $d \cdot e \equiv 1 \pmod{\phi(n)}$, i.e.,

$$7d \equiv 1 \pmod{20}$$

Testing: $7 \times 3 = 21 \equiv 1 \pmod{20}$

$$\boxed{d = 3}$$

Key summary:

- Public key: $(e, n) = (7, 33)$
- Private key: $(d, n) = (3, 33)$

(b) Sign Message $M=2$ (10 marks)

Alice signs using her **private key** ($d = 3, n = 33$):

$$S = M^d \bmod n = 2^3 \bmod 33 = 8 \bmod 33$$

$$\boxed{S = 8}$$

What the signature guarantees:

- Only Alice (holder of private key $d = 3$) can produce this signature.
 - Any alteration to M would produce a different S , detected during verification.
-

(c) Bob Verifies Alice's Signature (10 marks)

Bob has: message $M = 2$, signature $S = 8$, and Alice's **public key** ($e = 7, n = 33$).

Verification step: Compute $M' = S^e \bmod n$ and check $M' \stackrel{?}{=} M$.

$$M' = 8^7 \bmod 33$$

Computing step by step:

$$\begin{aligned} 8^2 &= 64 \equiv 64 - 33 = 31 \pmod{33} \\ 8^4 &= 31^2 = 961 \equiv 961 - 29 \times 33 = 961 - 957 = 4 \pmod{33} \\ 8^7 &= 8^4 \times 8^2 \times 8^1 = 4 \times 31 \times 8 \pmod{33} \\ 4 \times 31 &= 124 \equiv 124 - 3 \times 33 = 124 - 99 = 25 \pmod{33} \\ 25 \times 8 &= 200 \equiv 200 - 6 \times 33 = 200 - 198 = 2 \pmod{33} \end{aligned}$$

$$M' = 2 = M \checkmark$$

Conclusion: Since $M' = M$, Bob confirms:

1. **Authenticity** — the message was signed by the holder of private key d corresponding to Alice's public key e .
2. **Integrity** — the message has not been altered in transit (any change to M or S would cause $M' \neq M$).

The RSA property $M = (M^d)^e \bmod n$ holds because $de \equiv 1 \pmod{\phi(n)}$, so $M^{de} \equiv M^1 \equiv M \pmod{n}$ by Euler's theorem.

Question 4 — Consistent Hashing (25 marks)

Ring: hash values 0 to 1023 (clockwise).

Servers: S1@100, S2@300, S3@600, S4@900.

Data items and hash values:

Item	Hash
D1	50
D2	150
D3	500
D4	700
D5	850
D6	950

Rule: each data item is stored on the **first server clockwise** from its hash value.

(a) Initial Server Assignment (5 marks)

Ring clockwise order: S1@100 → S2@300 → S3@600 → S4@900 → (wrap) → S1@100

Data item	Hash	Next server clockwise	Assigned to
D1	50	50→100 = S1	S1
D2	150	150→300 = S2	S2
D3	500	500→600 = S3	S3
D4	700	700→900 = S4	S4
D5	850	850→900 = S4	S4
D6	950	950→(wrap)→100 = S1	S1

(b) Server S2 Removed (10 marks)

Removing **S2@300**. Remaining servers: **S1@100**, **S3@600**, **S4@900**.

Only data items previously assigned to S2 need to be redistributed. They move to the next server clockwise after S2's position (300), which is now **S3@600**.

Data item	Old server	New next clockwise from hash	New assignment
D1@50	S1	50→100 = S1	S1 (unchanged)
D2@150	S2	150→300 (gone)→600 = S3	S3 (moved)
D3@500	S3	500→600 = S3	S3 (unchanged)
D4@700	S4	700→900 = S4	S4 (unchanged)
D5@850	S4	850→900 = S4	S4 (unchanged)
D6@950	S1	950→(wrap)→100 = S1	S1 (unchanged)

Only D2 moves — from S2 to S3. All other assignments are unchanged. This demonstrates the key advantage of consistent hashing: **minimal data movement on server departure**.

(c) New Server S5 Added at Hash 200 (10 marks)

Adding **S5@200**. Servers in order: **S1@100**, **S5@200**, **S2@300**, **S3@600**, **S4@900**.

Only data items in the arc (100, 200] (clockwise from S1 to S5) move to S5.

Data item	Hash	New next clockwise	New assignment
D1@50	50	50→100 = S1	S1 (unchanged)
D2@150	150	150→200 = S5	S5 (moved from S2)
D3@500	500	500→600 = S3	S3 (unchanged)
D4@700	700	700→900 = S4	S4 (unchanged)
D5@850	850	850→900 = S4	S4 (unchanged)
D6@950	950	950→(wrap)→100 = S1	S1 (unchanged)

Only D2 moves — from S2 to the new S5. No other items are affected. This again demonstrates consistent hashing's property: **only $O(K/N)$ keys move on average when a server is added** (where K = number of data items, N = number of servers), far less than the $O(K)$ movement that simple modular hashing would require.